

软件工程大作业软件配置与运维文档

一、引言

ArtVision 智能艺术导览系统致力于为艺术爱好者提供便捷的艺术作品识别、艺术信息探索及交互式对话体验。为保障系统在开发、测试及生产环境的稳定运行与高效迭代，特制定本配置与运维文档，涵盖配置管理、版本控制、持续集成、部署策略及运维监控等关键环节。本文档面向系统管理员、开发人员及运维工程师，旨在提供标准化的操作规范与技术参考。

二、软件配置管理

2.1 系统架构与运行环境

ArtVision 是一个由个人开发者独立构建的艺术导览网站，采用简洁高效的技术架构实现核心功能。系统前端由静态 HTML/CSS/JavaScript 页面构成，通过 Tailwind CSS 实现响应式布局，确保在移动设备和桌面端均有良好表现；后端服务基于 Python Flask 框架开发，处理图片上传、艺术识别和对话交互等核心逻辑；AI 能力通过阿里云 Qwen-Turbo 多模态模型实现，开发者已申请测试用 API 密钥；数据存储完全基于本地文件系统，在项目根目录建立清晰的目录结构管理用户上传图片和识别结果缓存。整套系统部署在腾讯云轻量应用服务器上，选用基础配置(1 核 CPU/2GB 内存/50GB SSD)即可满足日均 500 次请求的负载需求，确保安全更新及时可靠。

2.2 安全配置管理

前端配置通过 Webpack 环境变量注入实现动态加载，在构建时确定特性开关状态，例如是否启用 3D 预览功能。系统启动阶段会执行配置完整性校验，对缺失的关键参数如 Qwen-Turbo 模型密钥立即抛出异常，防止配置不完整导致运行时故障。

2.3 服务器配置与初始化

服务器环境初始化需按步骤完成基础软件安装和目录规划。首先通过 SSH 连接云服务器，更新系统包索引后安装 Python 环境、Web 服务器和版本管理工具，具体命令包括执行 `sudo apt update` 刷新软件源，安装 `python3-pip` 提供

Python 包管理能力，配置 git 用于代码版本控制。接着在用户主目录创建 /home/artvision/ 作为项目根路径，其下建立三个核心子目录：app/ 存放前后端源代码，data/ 管理用户上传图片和识别结果，logs/ 收集系统运行日志。网络配置方面开放必要端口，通过 `sudo ufw allow 80/tcp` 启用 HTTP 访问，`sudo ufw allow 443/tcp` 支持 HTTPS 加密连接，`sudo ufw allow 5000/tcp` 开放 Flask 后端服务端口，最后激活防火墙规则。Python 虚拟环境创建在 /home/artvision/app/backend/venv 路径，激活后安装 Flask 框架及依赖库，形成隔离的运行环境。

三、日常维护操作规范

3.1 版本控制

项目采用 GitFlow 工作流进行代码版本管理，建立清晰的分支策略保障开发有序性。

主分支 main 仅用于生产环境发布，始终保持稳定可部署状态；开发分支 develop 作为日常集成主干；功能开发在 feature/前缀分支进行，每个功能点对应独立分支。

版本发布前创建 release 分支执行最终测试，通过后合并至 main 分支并打上语义化版本标签（如 v1.2.0）。团队约定提交信息必须明确描述变更类型和影响范围，例如 "fix(upload): 修复大文件上传超时问题"。配合预提交自动运行基础检查，拒绝包含语法错误或测试失败的代码提交，从流程上保障代码库质量。

每周执行一次分支清理，合并超过两周的停滞分支，保持版本库整洁高效。

3.2 持续集成

系统监控通过组合基础命令实现资源使用情况跟踪，定期执行 `htop` 命令观察 CPU 和内存占用，使用 `df -h` 检查磁盘空间利用率，特别关注上传目录的增长趋势。日志审查采用动态查看方式，通过 `tail -f /home/artvision/logs/access.log` 实时监控访问请求，使用 `grep "ERROR" /home/artvision/logs/error.log` 过滤错误信息及时发现问题。备份策略设计为每日定时任务，编写 shell 脚本将上传数据和识别结果打包压缩，通过 `scp` 传输到本地开发机备份，同时保留最近 7 天的备份副本。系统更新遵循谨慎原则，每月择机执行安全更新，操作前创建服务器快

照，更新后验证服务状态。API 密钥管理通过环境变量实现，在启动脚本中设置 `export QWEN_API_KEY=sk_test_****` 传递密钥，避免硬编码在源代码中。

3.3 故障应急处理

当服务不可用时首先通过系统命令进行初步诊断，执行 `systemctl status artvision` 检查后端进程运行状态，使用 `curl http://127.0.0.1:5000/health` 验证内部端点可达性，通过 `sudo journalctl -u artvision -n 50` 查看最近日志。针对常见问题制定解决方案：若出现上传功能异常则检查 `data/uploads/` 目录权限和磁盘空间；识别服务失败时验证 API 密钥有效性及配额状态；页面加载缓慢时检查错误日志并重启服务。服务恢复操作遵循标准化流程：先重启 Flask 服务进程，再重载配置，最后清理浏览器缓存测试功能。建立简单的监控提醒机制，配置服务器资源告警规则，当 CPU 连续 15 分钟超 80% 或磁盘使用超 90% 时发送邮件通知开发者，确保问题及时响应。

3.4 性能优化

性能优化从两个维度展开：前端资源启用浏览器缓存，为静态文件设置 30 天有效期，减少重复传输；后端服务实现识别结果缓存，相同图片哈希值直接返回历史结果，降低 API 调用频次。安全加固措施包括申请免费 SSL 证书启用 HTTPS 加密，配置自动续期避免中断；修改 Flask 服务仅监听本地回环地址，防止外部直接访问。

功能迭代路线规划：近期优先实现艺术家信息本地存储，减少外部依赖；中期计划增加访问统计功能，了解用户行为模式；远期考虑容器化部署方案，提升环境一致性。成本控制策略包括监控资源使用趋势，在业务量稳定后降配服务器规格；设置 API 调用频率限制，防止意外超额产生费用；利用云平台闲置计算资源优惠，降低运营成本。

四、结语

本方案通过标准化流程和自动化工具，构建可持续演进的技术运营体系。配置分层管理确保环境一致性，版本控制规范代码演进路径，持续集成快速反馈质量问题，立体化监控保障服务可用性。团队将定期评审运维指标并优化流程，为 ArtVision 用户提供稳定可靠的艺术探索体验。